The Evolution of Network Embeddings: DeepWalk to node2vec & NetMF

2026-08-26

Presenter: Min Hwang

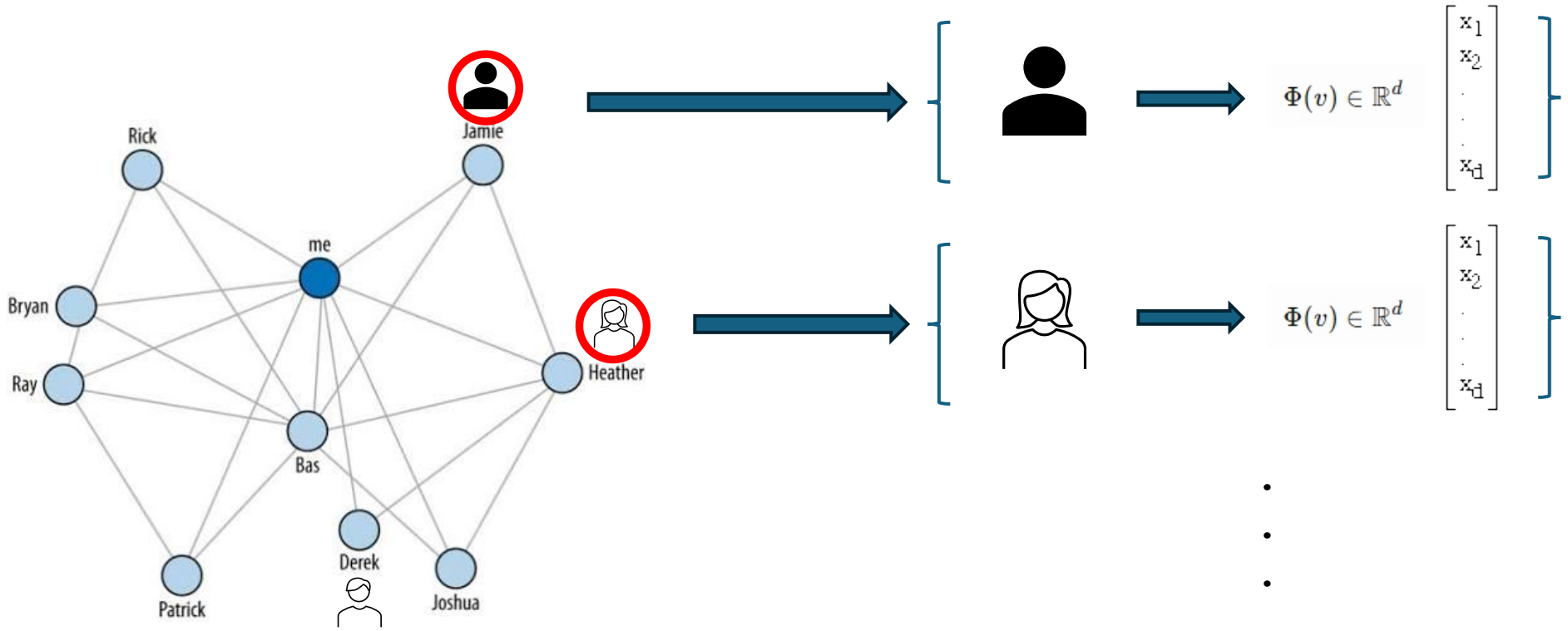
MINDS Lab, CAU

INDEX

- Graph Representation Learning
- DeepWalk
- node2vec
- NetMF
- Conclusion
- Appendix

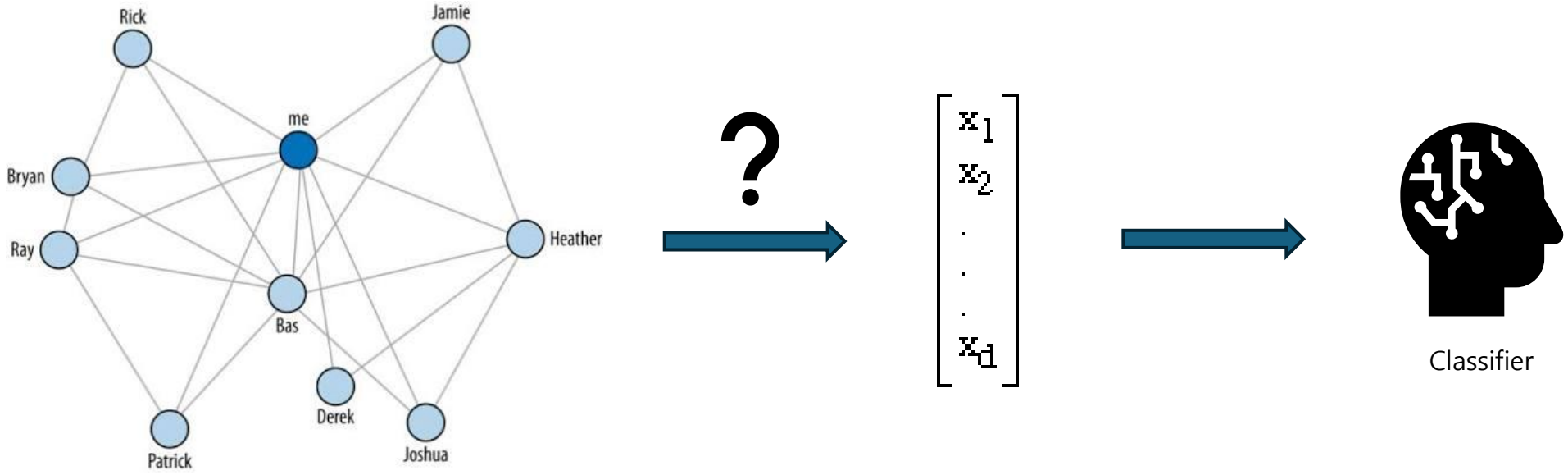
What is Graph Representation Learning?

- **Maps discrete graph structure into continuous, low-dimensional representations that downstream ML models can use.**



Why Do We Need GRL?

- How can we turn network structure into useful features for machine learning?



What Makes GRL Difficult?

- Irregular structure
- High dimensionality
- Multiple notions of similarity
- Scalability
- Dynamic structure

Three Views of Network Embedding

- **DeepWalk**
  - Can graph neighborhoods be learned like word contexts?
- **node2vec**
  - Can we control which neighborhoods are learned?
- **NetMF**
  - What matrix are these methods actually learning?

DeepWalk: Online Learning of Social Representations

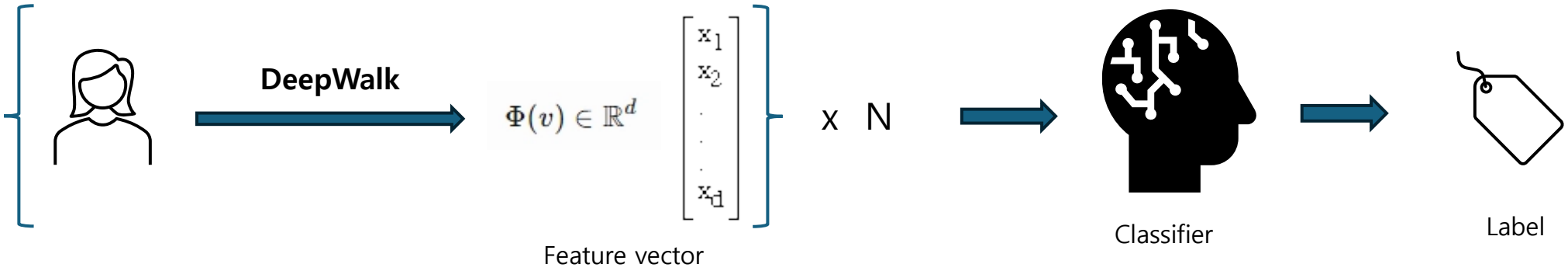
Bryan Perozzi[a], Rami Al-Rfou[a], Steven Skiena[a]

[a] *Stony Brook University, Stony Brook, New York*

KDD 2014 *(20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining)*

DeepWalk's Goal

- **Learn useful node representations directly from the network's structure**
  - Use relationships between nodes as the source of information
  - Learn representations useful for downstream prediction tasks

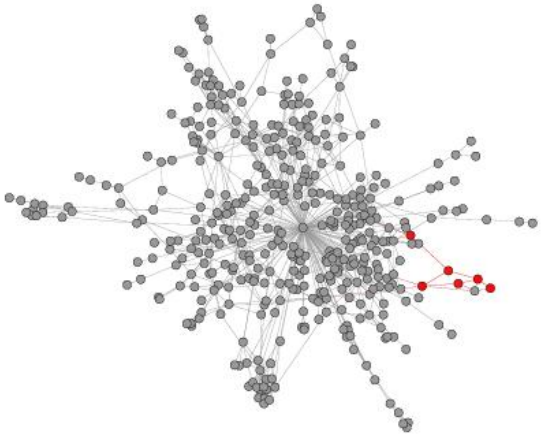


Traits

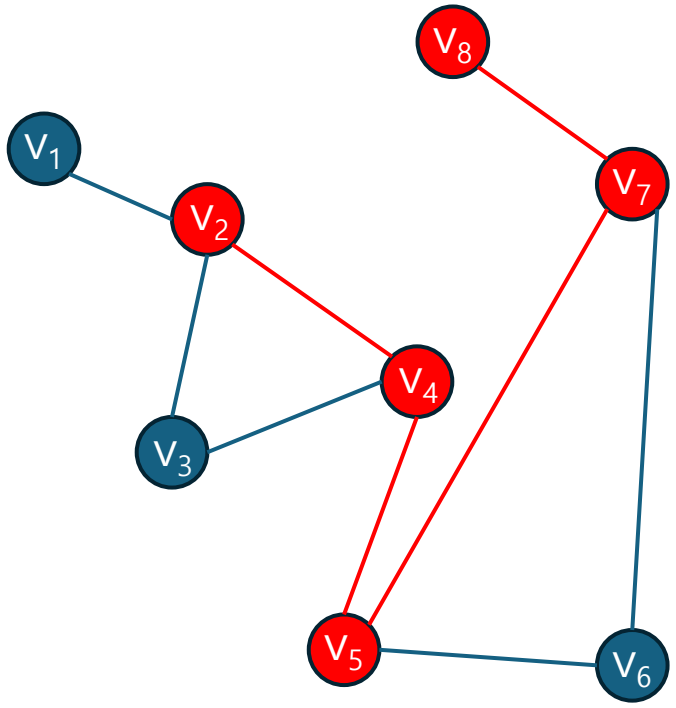
- Adaptability
- Community Awareness
- Low Dimensionality
- Continuity

How to Extract Structural Information?

- We want to efficiently extract structural information from the graph
- Random walks



(a) Random walk generation.



$$\mathcal{W}_{v_i} = (v_2, v_4, v_5, v_7, v_8)$$

Progress bar

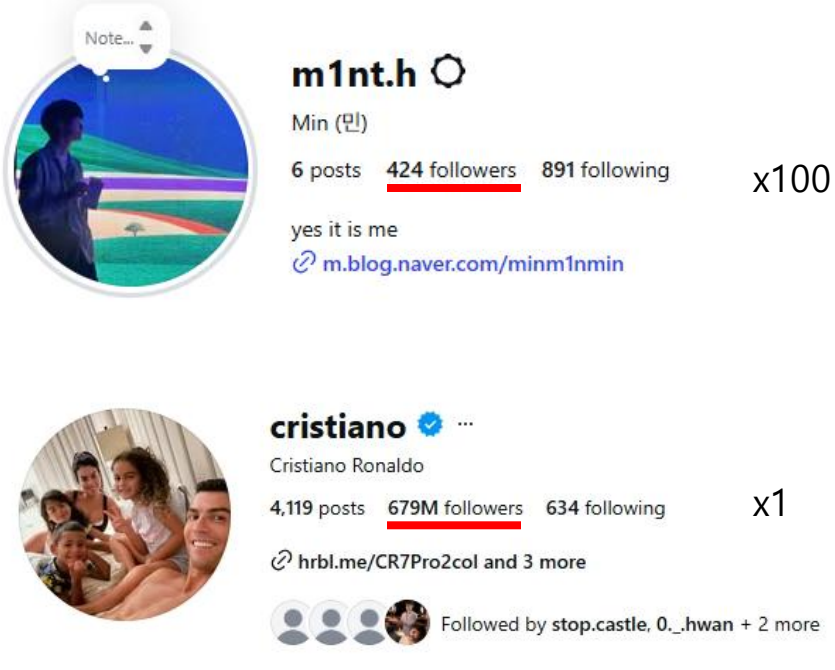
From Graphs to Language

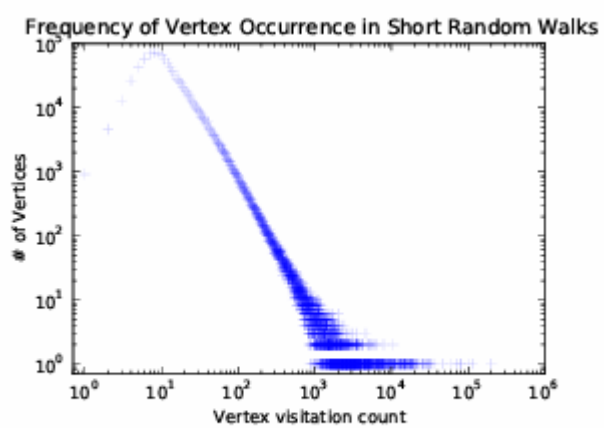
- **Random walks**
  - ☐ Scalable way to sample relational information from a graph
  - ☐ Easy to parallelize
  - ☐ Accomodate for local changes without requiring a global recomputation
- **Generate a sequence of vertices**

Power Law

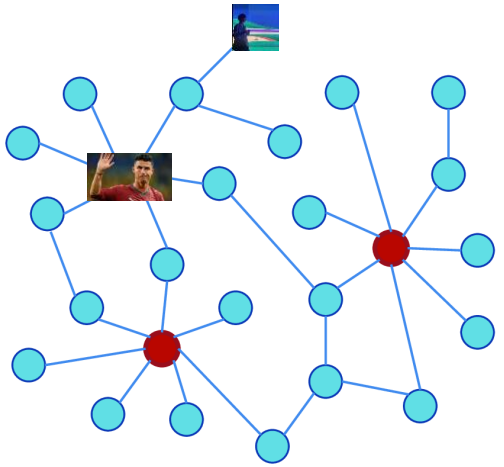
Real-world networks often have highly skewed degree distribution

- Random walks over these networks also follow power-law distribution



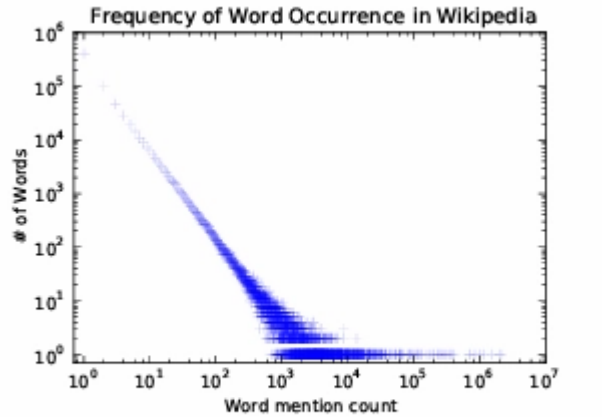


(a) YouTube Social Graph



From Graphs to Language

- **Word frequencies in natural language are also highly skewed**
  - Vertices in random walks show similar frequency characteristics
  - This motivates treating random walks as a form of language



(b) Wikipedia Article Text

$$P(v_i \mid v_1, \ldots, v_{i-1}),$$

word	$\leftrightarrow$	vertex
sentence	$\leftrightarrow$	random walk
corpus	$\leftrightarrow$	stream of random walks

From Walks to Context

- **Nearby vertices in a random walk define each vertex's context**

$$\mathcal{W}_{v_i} = (v_1, v_2, v_4, v_5, v_7, v_8, v_3)$$

$$w = 2$$

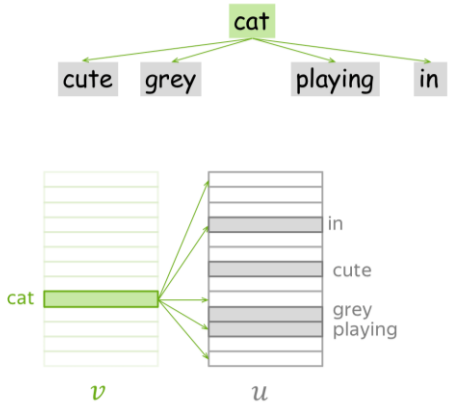
$$v_1 \quad \underbrace{v_2 \quad v_4}_{\text{context}} \quad \boxed{v_5} \quad \underbrace{v_7 \quad v_8}_{\text{context}} \quad v_3$$

$$\underbrace{v_5}_{\text{center}} \longrightarrow \underbrace{\{v_2, v_4, v_7, v_8\}}_{\text{context}}$$

Skip-Gram

- **Use center item to predict its surrounding context**

… I saw a cute grey cat playing in the garden …



$$\underbrace{v_5}_{\text{center}} \xrightarrow{\Phi} \Phi(v_5) \longrightarrow \underbrace{\{v_2, v_4, v_7, v_8\}}_{\text{context}}$$

$$\max_{\Phi} P\left(\{v_2, v_4, v_7, v_8\} \mid \Phi(v_5)\right)$$

Objective

Hierarchical Softmax

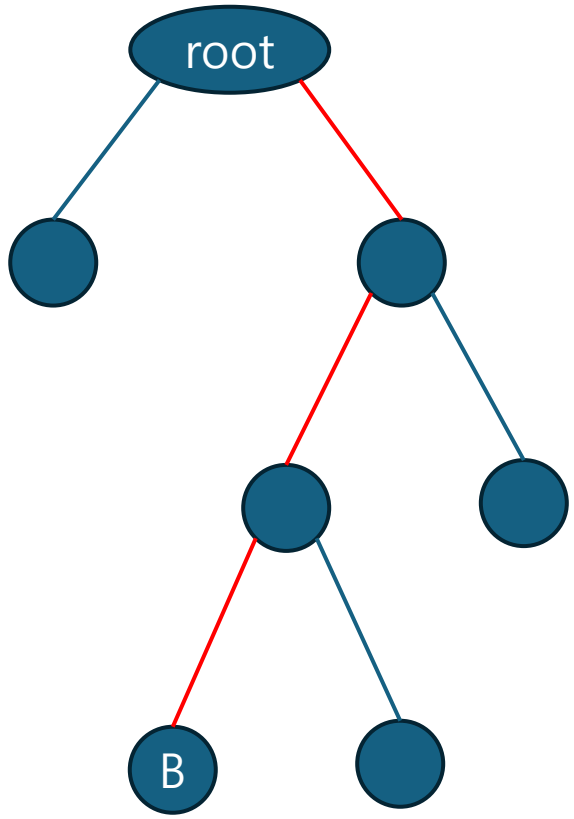
Traditional Softmax is expensive

- Use Hierarchical Softmax
- Create binary decision tree
- Vertices of the graph are leaf nodes
- Time complexity: O(|V|) ➔ O(log|V|)

$$P(u \mid v) = \frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{x \in V} \exp(\mathbf{z}_x^\top \mathbf{z}_v)}$$

Traditional Softmax

expensive!!!



$$\Pr(u_k \mid \Phi(v_j)) = \prod_{l=1}^{\lceil \log |V| \rceil} \Pr(b_l \mid \Phi(v_j)).$$

$$b_1 = right, \quad b_2 = left \quad b_3 = left$$

$$P(\text{right at root} \mid \Phi(v_j)) \times P(\text{left at node 2} \mid \Phi(v_j)) \times P(\text{left at node 3} \mid \Phi(v_j)).$$

Experiments

Datasets

- BlogCatalog, Flickr, YouTube

Baseline Methods

- Spectral Clustering, Modularity, EdgeCluster, wvRN, Majority

DeepWalk Setting

- γ = 80, w = 10, d = 128
- One-vs-rest logistic regression for final classification

Name	BLOGCATALOG	FLICKR	YOUTUBE
$\|V\|$	10,312	80,513	1,138,499
$\|E\|$	333,983	5,899,882	2,990,443
$\|\mathcal{Y}\|$	39	195	47
Labels	Interests	Groups	Groups

Results

	% Labeled Nodes	10%	20%	30%	40%	50%	60%	70%	80%	90%
Micro-F1(%)	DeepWalk	**36.00**	**38.20**	**39.60**	**40.30**	**41.00**	**41.30**	41.50	41.50	42.00
	SpectralClustering	31.06	34.95	37.27	38.93	39.97	40.99	**41.66**	**42.42**	**42.62**
	EdgeCluster	27.94	30.76	31.85	32.99	34.12	35.00	34.63	35.99	36.29
	Modularity	27.35	30.74	31.77	32.97	34.09	36.13	36.08	37.23	38.18
	wvRN	19.51	24.34	25.62	28.82	30.37	31.81	32.19	33.33	34.28
	Majority	16.51	16.66	16.61	16.70	16.91	16.99	16.92	16.49	17.26
Macro-F1(%)	DeepWalk	**21.30**	**23.80**	25.30	26.30	27.30	27.60	27.90	28.20	28.90
	SpectralClustering	19.14	23.57	**25.97**	**27.46**	**28.31**	**29.46**	**30.13**	**31.38**	**31.78**
	EdgeCluster	16.16	19.16	20.48	22.00	23.00	23.64	23.82	24.61	24.92
	Modularity	17.36	20.00	20.80	21.85	22.65	23.41	23.89	24.20	24.97
	wvRN	6.25	10.13	11.64	14.24	15.86	17.18	17.98	18.86	19.57
	Majority	2.52	2.55	2.52	2.58	2.58	2.63	2.61	2.48	2.62

Table 2: Multi-label classification results in BlogCatalog

	% Labeled Nodes	1%	2%	3%	4%	5%	6%	7%	8%	9%	10%
Micro-F1(%)	DeepWalk	**32.4**	**34.6**	**35.9**	**36.7**	**37.2**	**37.7**	**38.1**	**38.3**	**38.5**	**38.7**
	SpectralClustering	27.43	30.11	31.63	32.69	33.31	33.95	34.46	34.81	35.14	35.41
	EdgeCluster	25.75	28.53	29.14	30.31	30.85	31.53	31.75	31.76	32.19	32.84
	Modularity	22.75	25.29	27.3	27.6	28.05	29.33	29.43	28.89	29.17	29.2
	wvRN	17.7	14.43	15.72	20.97	19.83	19.42	19.22	21.25	22.51	22.73
	Majority	16.34	16.31	16.34	16.46	16.65	16.44	16.38	16.62	16.67	16.71
Macro-F1(%)	DeepWalk	**14.0**	17.3	**19.6**	**21.1**	**22.1**	**22.9**	**23.6**	**24.1**	**24.6**	**25.0**
	SpectralClustering	13.84	**17.49**	19.44	20.75	21.60	22.36	23.01	23.36	23.82	24.05
	EdgeCluster	10.52	14.10	15.91	16.72	18.01	18.54	19.54	20.18	20.78	20.85
	Modularity	10.21	13.37	15.24	15.11	16.14	16.64	17.02	17.1	17.14	17.12
	wvRN	1.53	2.46	2.91	3.47	4.95	5.56	5.82	6.59	8.00	7.26
	Majority	0.45	0.44	0.45	0.46	0.47	0.44	0.45	0.47	0.47	0.47

Table 3: Multi-label classification results in Flickr

	% Labeled Nodes	1%	2%	3%	4%	5%	6%	7%	8%	9%	10%
Micro-F1(%)	DeepWalk	**37.95**	**39.28**	**40.08**	**40.78**	**41.32**	**41.72**	**42.12**	**42.48**	**42.78**	**43.05**
	SpectralClustering	—	—	—	—	—	—	—	—	—	—
	EdgeCluster	23.90	31.68	35.53	36.76	37.81	38.63	38.94	39.46	39.92	40.07
	Modularity	—	—	—	—	—	—	—	—	—	—
	wvRN	26.79	29.18	33.1	32.88	35.76	37.38	38.21	37.75	38.68	39.42
	Majority	24.90	24.84	25.25	25.23	25.22	25.33	25.31	25.34	25.38	25.38
Macro-F1(%)	DeepWalk	**29.22**	**31.83**	**33.06**	**33.90**	**34.35**	**34.66**	**34.96**	**35.22**	**35.42**	**35.67**
	SpectralClustering	—	—	—	—	—	—	—	—	—	—
	EdgeCluster	19.48	25.01	28.15	29.17	29.82	30.65	30.75	31.23	31.45	31.54
	Modularity	—	—	—	—	—	—	—	—	—	—
	wvRN	13.15	15.78	19.66	20.9	23.31	25.43	27.08	26.48	28.33	28.89
	Majority	6.12	5.86	6.21	6.1	6.07	6.19	6.17	6.16	6.18	6.19

Table 4: Multi-label classification results in YouTube

node2vec: Scalable Feature Learning for Networks

Aditya Grover[a], Jure Leskovec[a]

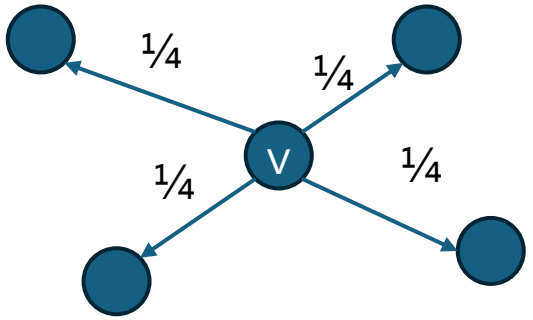
[a] *Stanford University, Stanford, California*

KDD 2016 *(22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining)*

DeepWalk's Limitation

DeepWalk uses a fixed, unbiased random-walk strategy

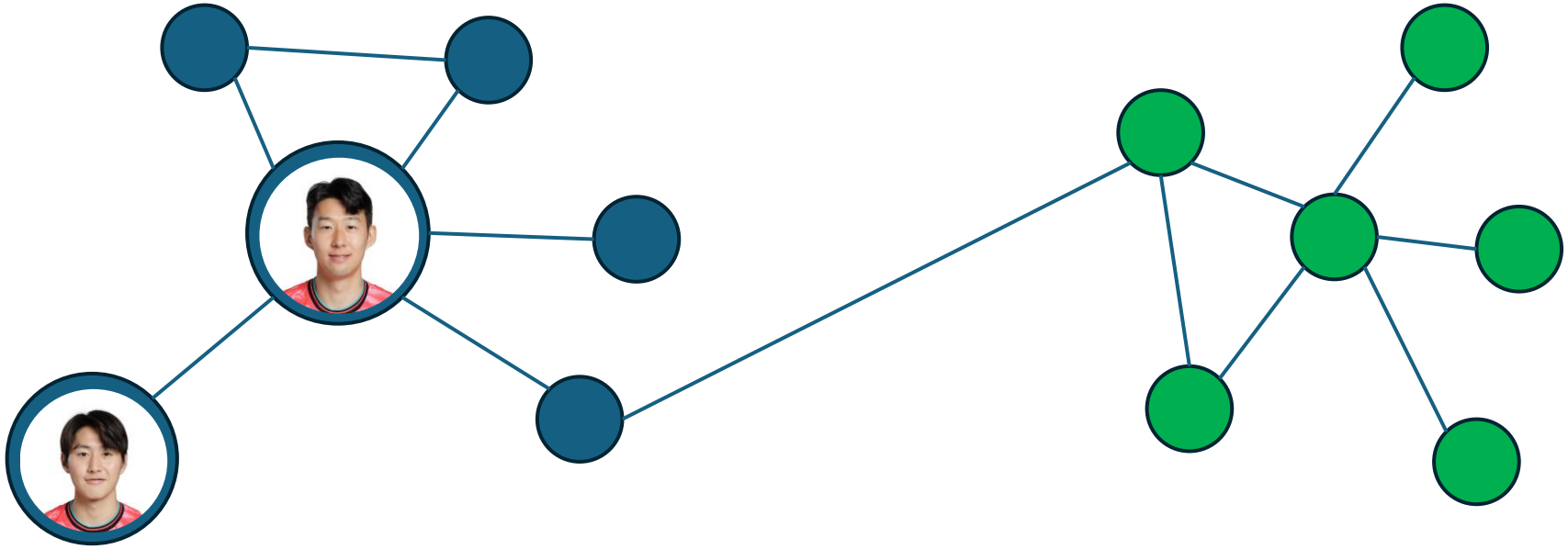
- Cannot flexibly adapt to different notions of node similarity



Node Similarity	
Homophily	Structural Equivalence
i.e. friends, family, co-authors	i.e. team captains on different teams
Real networks contain BOTH types	

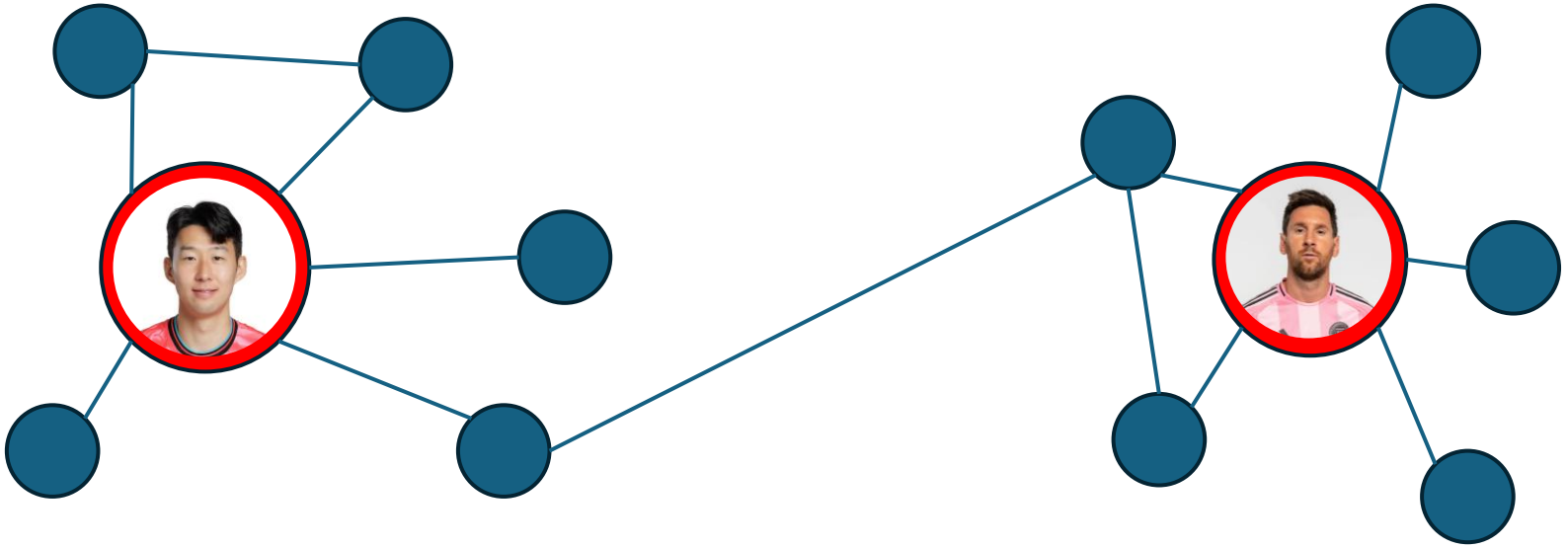
Different Tasks Need Different Contexts

- **Same community (homophily)**



Different Tasks Need Different Contexts

- **Same role (structural equivalence)**



Two Ways to Explore the Graph

- **Different exploration strategies expose different neighborhoods**

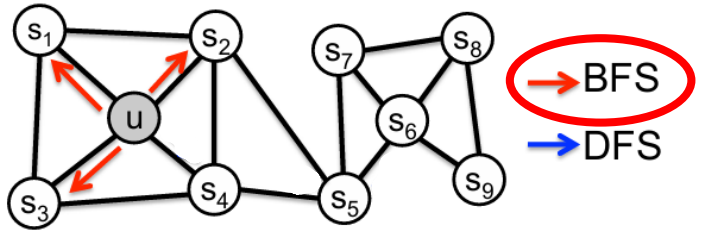


Figure 1: BFS and DFS search strategies from node u ($k = 3$).

- Stays close to the starting node
- Samples the local neighborhood repeatedly
- Tends to emphasize homophily

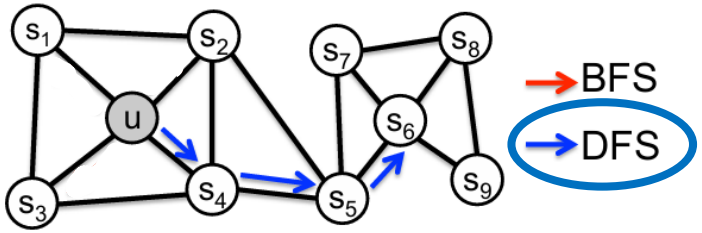


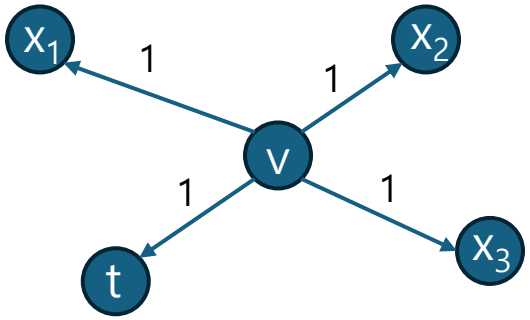
Figure 1: BFS and DFS search strategies from node u ($k = 3$).

- Moves farther away from the starting node
- Explores different regions
- Can reveal structurally similar nodes

node2vec's Core Idea

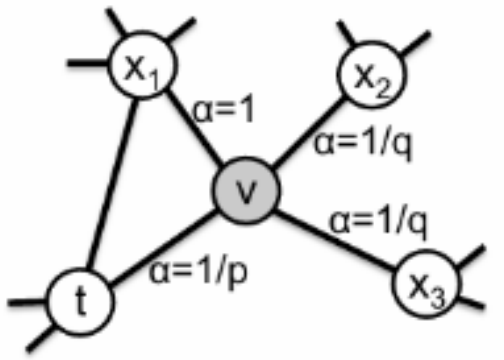
Make the random walk strategy flexible

- Biased random walk
- Probability of taking the next step depends not only on where we are now, but also on **where we came from**
- Return, stay local, or move outwards



DeepWalk's unbiased random walk

$$\alpha_{pq}(t,x) = \begin{cases} 1/p & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ 1/q & \text{if } d_{tx} = 2 \end{cases}$$

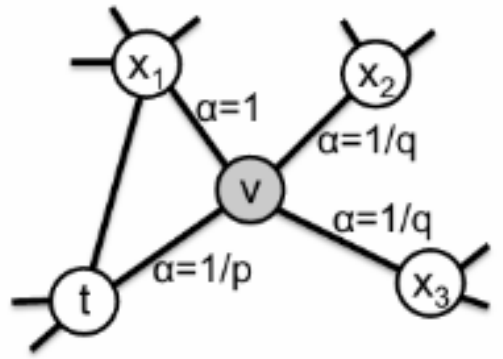


node2vec's biased random walk

Controlling the Random Walk

- **How does node2vec actually implement “flexibility”?**

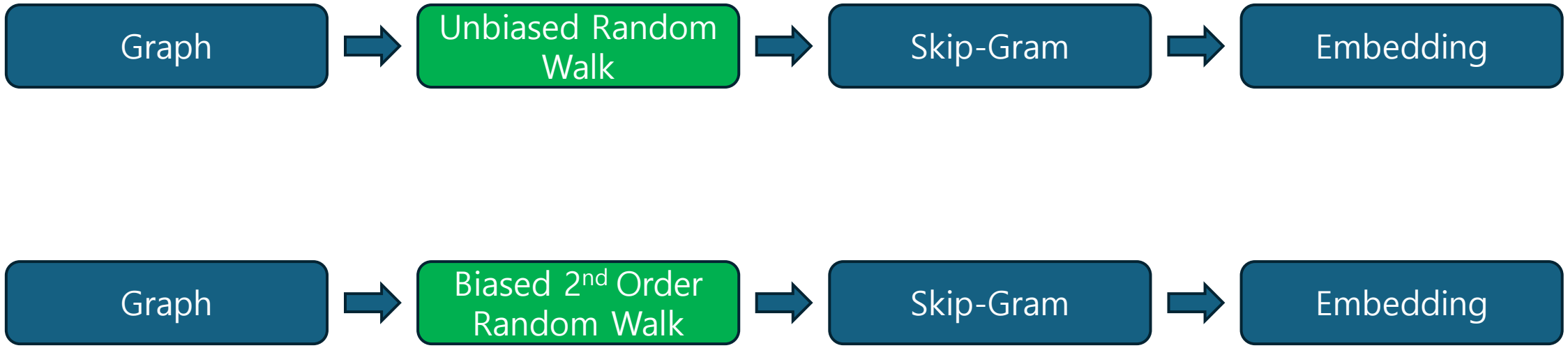
$$\alpha_{pq}(t,x) = \begin{cases} 1/p & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ 1/q & \text{if } d_{tx} = 2 \end{cases}$$



Parameter	< 1	= 1	> 1
p	Favors returning	Neutral	Discourages returning
q	Favors outward exploration	Neutral	Favors local exploration

From DeepWalk to node2vec

- **Same overall framework, different context sampler**



Training with Negative Sampling

- **node2vec trains observed node-context pairs against randomly sampled non-context pairs**
  - Same Skip-Gram idea, different optimization from original DeepWalk's hierarchical softmax

$$Pr(n_i|f(u)) = \frac{\exp(f(n_i) \cdot f(u))}{\sum_{v \in V} \exp(f(v) \cdot f(u))}.$$

Softmax normalization term is too expensive to calculate
➔ Approximate using negative sampling!

(C, B) → $\text{score}(C, B) \uparrow$

Real center-context pair

$(C, X_1), \quad (C, X_2), \quad (C, X_3)$ → $\text{score}(C, X_i) \downarrow$

Randomly sampled negative pairs

Experiments & Results

Tasks

- Multi-label classification (BlogCatalog, PPI, Wikipedia)
- Link Prediction (Facebook, PPI, arXiv)

Baselines

- DeepWalk, LINE, Spectral Clustering

Findings

- Up to 26.7% improvement in classification
- Up to 12.6% improvement in link prediction
- Robust to missing edges & noise
- Approximately linearly scalable
- Hadamard operator achieved best link prediction performance

Algorithm	**Dataset**		
	BlogCatalog	PPI	Wikipedia
Spectral Clustering	0.0405	0.0681	0.0395
DeepWalk	0.2110	0.1768	0.1274
LINE	0.0784	0.1447	0.1164
node2vec	**0.2581**	**0.1791**	**0.1552**
node2vec settings (p,q)	0.25, 0.25	4, 1	4, 0.5
Gain of *node2vec* [%]	**22.3**	**1.3**	**21.8**

Op	**Algorithm**	**Dataset**		
		Facebook	PPI	arXiv
	Common Neighbors	0.8100	0.7142	0.8153
	Jaccard's Coefficient	0.8880	0.7018	0.8067
	Adamic-Adar	0.8289	0.7126	0.8315
	Pref. Attachment	0.7137	0.6670	0.6996
(a)	Spectral Clustering	0.5960	0.6588	0.5812
	DeepWalk	0.7238	0.6923	0.7066
	LINE	0.7029	0.6330	0.6516
	node2vec	0.7266	0.7543	0.7221
(b)	Spectral Clustering	0.6192	0.4920	0.5740
	DeepWalk	**0.9680**	0.7441	0.9340
	LINE	0.9490	0.7249	0.8902
	node2vec	**0.9680**	**0.7719**	**0.9366**
(c)	Spectral Clustering	0.7200	0.6356	0.7099
	DeepWalk	0.9574	0.6026	0.8282
	LINE	0.9483	0.7024	0.8809
	node2vec	0.9602	0.6292	0.8468
(d)	Spectral Clustering	0.7107	0.6026	0.6765
	DeepWalk	0.9584	0.6118	0.8305
	LINE	0.9460	0.7106	0.8862
	node2vec	0.9606	0.6236	0.8477

Conclusions

- Flexibility is useful
- Embeddings are useful for more than one task
- node2vec improves on rigid sampling
- The method is practical

Network Embedding as Matrix Factorization: Unifying DeepWalk, LINE, PTE, and node2vec

Jiezhong Qiu[a], Yuxiao Dong[b], Hao Ma[b], Jian Li[a], Kuansan Wang[b], Jie Tang[a]

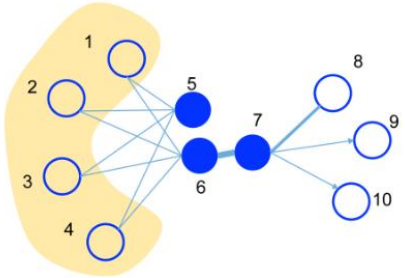
[a] *Tsinghua University, Beijing, China*
[b] *Microsoft Research, Redmond, Washington*

WSDM 2018 *(11th ACM International Conference on Web Search and Data Mining)*

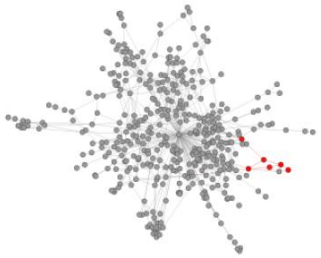
Motivation

What are DeepWalk, LINE, PTE, and node2vec actually learning?

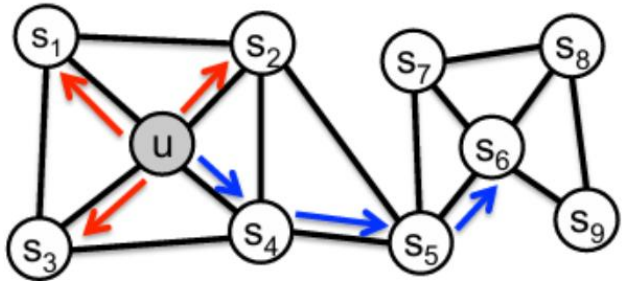
- What do they deem as “network context”?



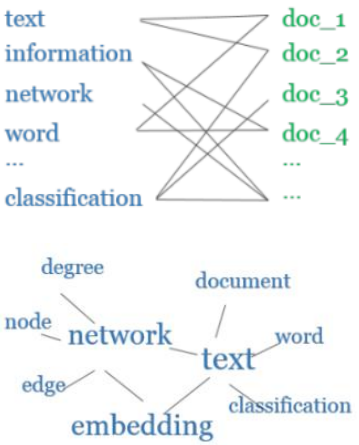
LINE: Direct neighbors



DeepWalk: Nearby nodes in unbiased random walks



node2vec: Nearby nodes in biased random walks

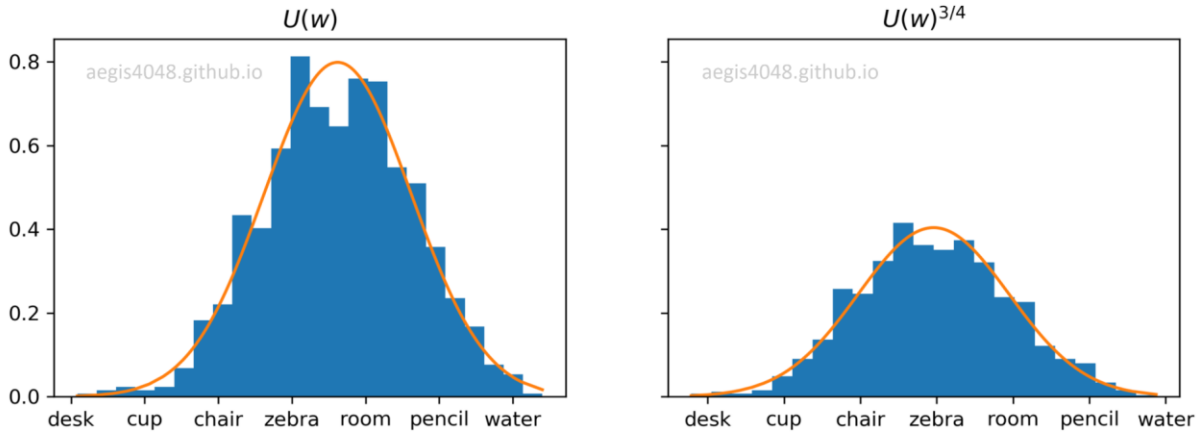


PTE: Word-word, document-word, label-word relations

DeepWalk Learns Indirectly via Sampling

- **DeepWalk with SGNS learns its target indirectly through sampling**
  - Graph ➔ **random walk** ➔ sampled co-occurrences ➔ **SGNS** ➔ embeddings
  - Why not calculate the expected matrix directly?

x 49 x 51



Negative sampling distribution is inherently noisy

What Does DeepWalk Give to SGNS?

- **DeepWalk converts random walk windows into a multiset of positive vertex–context pairs**

$$\mathcal{W}_{v_i} = (v_1, v_2, v_4, v_5, v_7, v_8, v_3)$$

$$w = 2$$

$$v_1 \quad \underbrace{v_2 \quad v_4}_{\text{context}} \quad \boxed{v_5} \quad \underbrace{v_7 \quad v_8}_{\text{context}} \quad v_3$$

$$\mathcal{D} = \{(v_5, v_2), (v_5, v_4), (v_5, v_7), (v_5, v_8), \ldots\}$$

$\#(w, c)$

Number of times the pair (w,c) occurs in D

$\#(w) \quad \#(c)$

How often w appears as center and c as context

What Does SGNS Do with D?

- **Each observed pair is contrasted against sampled negative contexts**
  - Observed pair ➔ pull together
  - Negative samples ➔ push apart

$(w, c) \in \mathcal{D}$

Positive pairs

$x_w^\top y_c \uparrow$

$(w, c_1^-), (w, c_2^-), \ldots, (w, c_b^-)$

Sampled negative pairs

$x_w^\top y_{c_i^-} \downarrow$

$$\log \sigma \left(x_w^\top y_c \right) + \sum_{i=1}^{b} \log \sigma \left(-x_w^\top y_{c_i^-} \right)$$

Maximize the objective

Expected Negative Frequency

- **Individual negative samples are random, but their expected frequency is predictable**

$$\#(w, c)$$

Positive evidence

$$\frac{b\#(w)\#(c)}{|\mathcal{D}|}$$

Expected negative evidence
(b: negatives sampled per positive pair)

What Matrix Is SGNS Implicitly Learning?

Compare positive count vs expected negative count

- PMI measures how much more often two items co-occur than expected if they were independent.

$$\frac{\#(w,c)}{\frac{b\,\#(w)\#(c)}{|\mathcal{D}|}} \xrightarrow{\log} \log\left(\frac{\#(w,c)|\mathcal{D}|}{\#(w)\#(c)}\right) - \log b \Rightarrow M_{ij} = \mathrm{PMI}(i,j) - \log b$$

Positive evidence to expected negative evidence ratio

Shifted PMI

From Counts to Transition Probabilities

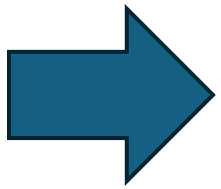
- **Replace sampled co-occurrence frequencies with their expected values under the random walk**

$$P = D^{-1}A$$

P_{ij} : Probability of moving from v_i to v_j in one random walk step

$$(P^r)_{ij} = \Pr(v_{t+r} = v_j \mid v_t = v_i)$$

Probability of reaching v_j from v_i after r steps



$$\frac{1}{T}\sum_{r=1}^{T}(P^r)_{ij}$$

Average reachability across the context window

The DeepWalk Matrix

- **Combine the expected random-walk statistics into the matrix SGNS implicitly factorizes**
  - Pr: r-step random-walk proximity
  - T: context-window size
  - D−1: degree/frequency correction
  - b: number of negative samples
  - vol(G): graph-volume normalization

$$M = \frac{\mathrm{vol}(G)}{bT}\left(\sum_{r=1}^{T} P^r\right) D^{-1}$$

$$\mathrm{vol}(G) = \sum_{v \in V} d_v$$

$$M' = \log M$$

Truncating the DeepWalk Matrix

- **Negative shifted-PMI values are truncated to zero before factorization**
  - Treat all instances of a context vertex appearing less often than the negative noise baseline would predict as absent connections

$$M_{wc} = \log\left[\frac{\mathrm{vol}(G)}{bT}\left(\sum_{r=1}^{T}(P^r)_{wc}\right)\frac{1}{d_c}\right] \quad \Rightarrow \quad M'_{wc} = \max\left(M_{wc}, 0\right)$$

DeepWalk matrix entry (w,c)

Truncated matrix entry (w,c)

From Matrix to Embeddings

- **Factorize M′ and keep only the top d singular components**

$$M' \in \mathbb{R}^{n \times n}$$

Shifted-PMI Matrix

$$M' \approx U_d \Sigma_d V_d^\top$$

Truncated SVD

$$X = U_d \Sigma_d^{1/2}$$

Low-Dimensional Representation

$$X \in \mathbb{R}^{n \times d} \qquad d \ll n$$

Node Embeddings

The NetMF Pipeline

- **Replace DeepWalk's implicit stochastic learning with explicit matrix factorization**

DeepWalk

NetMF

Implicit factorization → Explicit factorization

The Cost of Explicit Factorization

- **The DeepWalk matrix can become dense and expensive to factorize**
  - Space complexity: can approach $O(n^2)$
  - Direct factorization: expensive for large n
  - Explicit factorization improves interpretability, but sacrifices scalability

"Small-world intuition
92% of Facebook user pairs were within five degrees of separation."

Efficient Multi-Step Approximation

- **Use eigen-decomposition to avoid repeatedly computing large matrix powers**
  - For large T, keep only the top h eigenpairs

$$S = D^{-1/2} A D^{-1/2}$$

Symmetric Normalized Adjacency Matrix

$$S = U \Lambda U^{\top}$$

Eigen-decomposition of S

$$\sum_{r=1}^{T} S^r = U \left(\sum_{r=1}^{T} \Lambda^r \right) U^{\top}$$

Multi-Step Sum in the Spectral Domain

The Effect of Window Size T

- **T controls how much local vs broader graph structure is emphasized**
  - T is the skip-gram window
  - Larger T emphasize smoother, large-scale structures (such as communities)
  - But with that, sensitivity to local rapidly varying structures is blurred

$$\left(\frac{1}{T}\sum_{r=1}^{T}\boldsymbol{P}^r\right)\boldsymbol{D}^{-1} = \left(\boldsymbol{D}^{-1/2}\right)\left(\boldsymbol{U}\left(\frac{1}{T}\sum_{r=1}^{T}\boldsymbol{\Lambda}^r\right)\boldsymbol{U}^\top\right)\left(\boldsymbol{D}^{-1/2}\right)$$

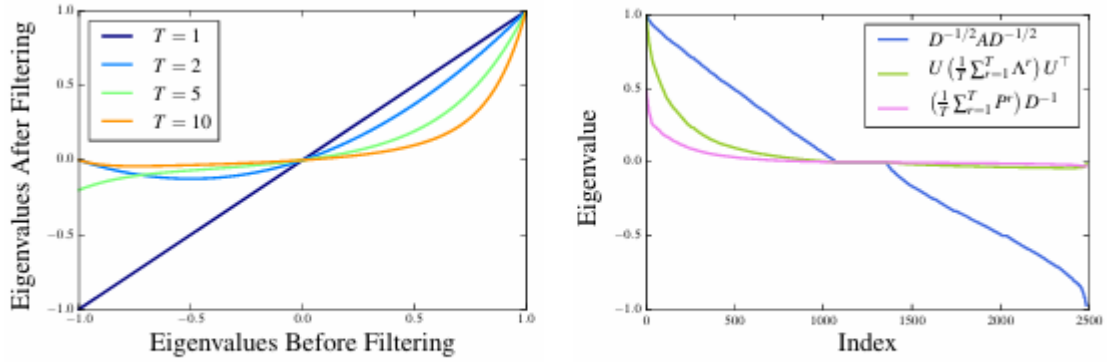
$$f_T(x) = \frac{1}{T}\sum_{r=1}^{T} x^r$$

When $T = 2$,

$$f_2(0.5) = \frac{0.5 + 0.25}{2} = 0.375.$$

When $T = 10$,

$$f_{10}(0.5) = \frac{0.5 + 0.25 + \cdots + 0.5^{10}}{10} \approx 0.1.$$



(a) (b)

Experiments & Results

Datasets

- BlogCatalog, PPI, Wikipedia, Flickr

Comparisons

- NetMF(T=1) vs LINE
- NetMF(T=10) vs DeepWalk

Algorithm	BlogCatalog (10%)		PPI (10%)		Wikipeida (10%)		Flickr (1%)	
	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1
LINE (2nd)	23.64	13.91	10.94	9.04	41.77	**9.72**	25.18	9.32
NetMF ($T = 1$)	33.04	14.86	16.01	12.10	**49.90**	9.25	23.87	6.44
Relative Gain of NetMF ($T = 1$)	39.76	6.83%	46.34%	33.85%	19.46%	-4.84%	-5.20%	-30.90%
DeepWalk	29.32	18.38	12.05	10.29	36.08	8.38	26.21	12.43
NetMF ($T = 10$)	**38.36**	**22.90**	**18.16**	**14.32**	46.21	8.38	**29.95**	**13.50**
Relative Gain of NetMF ($T = 10$)	30.83%	24.59%	50.71%	39.16%	28.08%	0.00%	14.27%	8.93%

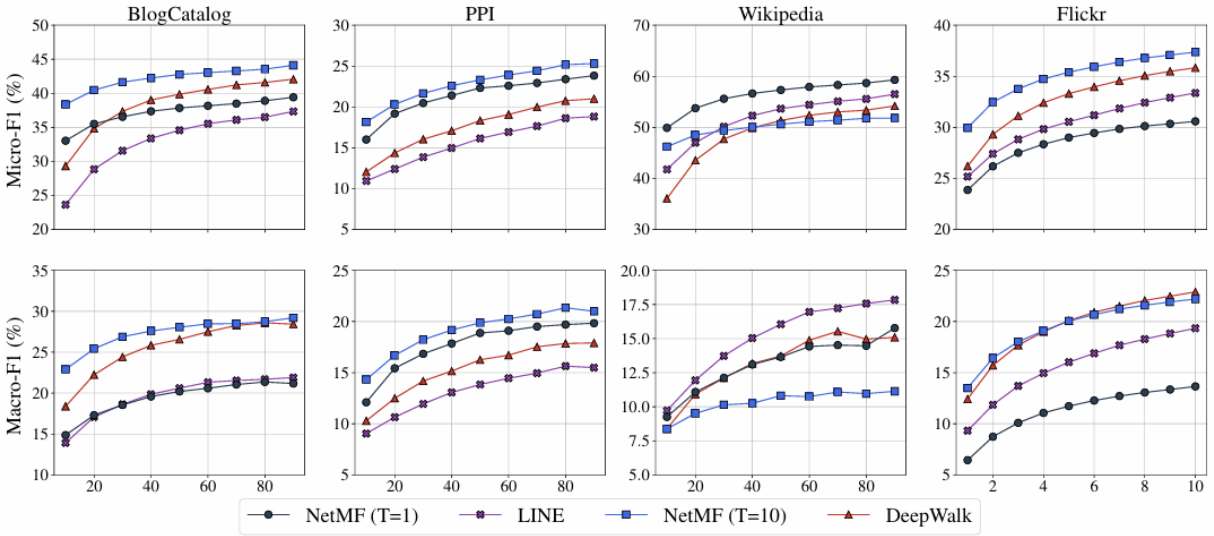


Figure 2: Predictive performance on varying the ratio of training data. The x-axis represents the ratio of labeled data (%), and the y-axis in the top and bottom rows denote the Micro-F1 and Macro-F1 scores respectively.

Results

- NetMF avoids the sampling error of DeepWalk, producing stronger embeddings

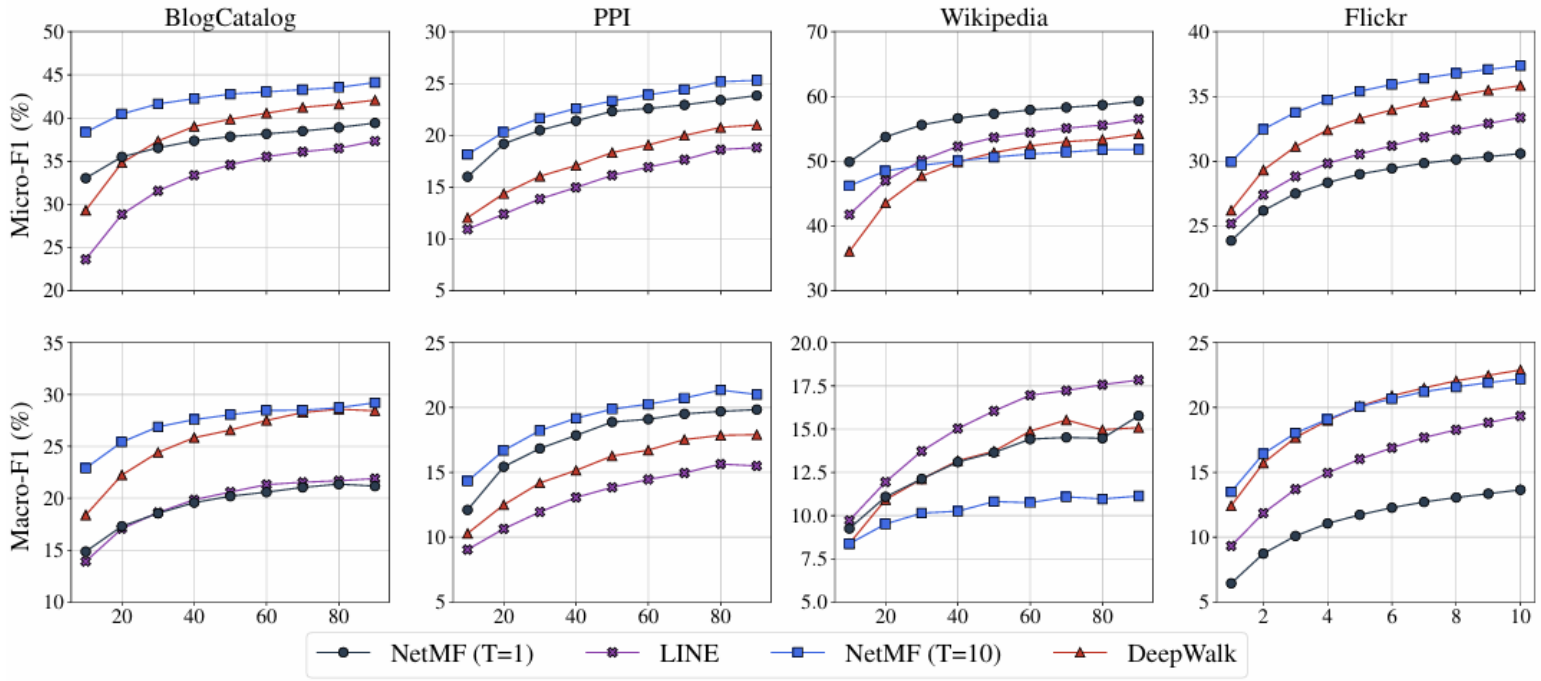


Figure 2: Predictive performance on varying the ratio of training data. The x-axis represents the ratio of labeled data (%), and the y-axis in the top and bottom rows denote the Micro-F1 and Macro-F1 scores respectively.

Conclusion

- DeepWalk — samples network context with unbiased random walks
- node2vec — makes that context sampling tunable
- NetMF — reveals the matrix these methods implicitly factorize

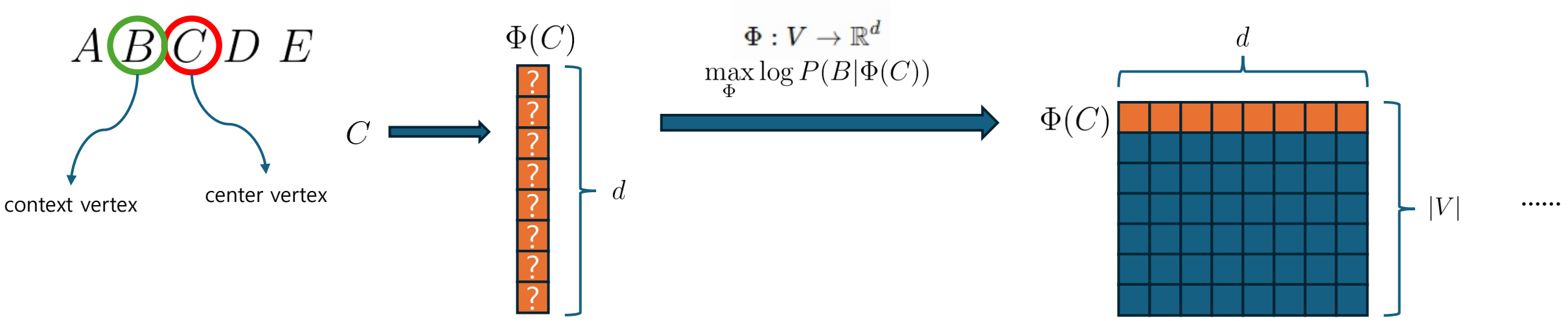
Network embedding evolved from sampling context, to controlling contex t, to understanding context mathematically.

Q&A

Appendix (DeepWalk Algorithm)

Learn a d-dimensional vector for each vertex “v”

- Ex: center vertex C & context vertex B
- Randomly initialize vectors
- Maximize P(B|Φ(C))
- But Φ(C) isn’t a probability distribution!!



Appendix (The DeepWalk Matrix)

- **Combine the expected random-walk statistics into the matrix SGNS imp licitly factorizes**

$$log\left(\frac{\#(w,c)|\mathcal{D}|}{\#(w)\cdot\#(c)}\right)-logb \longrightarrow M_{w,c}=log\frac{P_{context}(c|w)}{bP(c)} \longleftarrow P(c)=\frac{d_c}{vol(G)}$$

The building blocks of "M"

$$P_{context}(c|w)=\frac{1}{T}\sum_{r=1}^{T}(P^r)_{w,c}$$

$$M=\frac{vol(G)}{bT}\left(\sum_{r=1}^{T}P^r\right)D^{-1} \longleftarrow M_{w,c}=log\left(\frac{vol(G)}{bT}\left(\sum_{r=1}^{T}(P^r)_{w,c}\right)\frac{1}{d_c}\right)$$

$(P=D^{-1}A)$